

CSE-307: Operating Systems

Term Paper Brief - Section B

“Learning-Augmented OS Heuristics: Classical Algorithms Meet Adaptive Prediction”

1. Overview

This term paper is a small research-style exercise that connects a classical Operating Systems topic with a lightweight, data-driven (machine learning) component. The goal is not to build a large system, but to (a) implement a standard OS algorithm correctly, (b) add a simple adaptive or learned layer on top of it, and (c) study how the algorithm behaves when the underlying workload shifts partway through — a condition real operating systems constantly face once deployed.

You will need approximately 3–4 hours of focused work, including implementation, experimentation, and write-up.

2. Format

- Work individually
- Each student selects exactly ONE track from Section 4.
- You may use standard libraries for the learned component (e.g., scikit-learn, NumPy, pandas for Python).
- Using an AI coding assistant (ChatGPT, Claude, Copilot, etc.) for implementation help is allowed, but the experimental design, results, and analysis must be your own. Briefly disclose any AI assistance used in your README.
- Submission includes a GitHub repository (code) and a short written report (Section 6).

3. General Theme

Every track follows the same underlying structure:

- Implement one or more classical OS algorithms covered in the syllabus.
- Add a simple, lightweight learned or adaptive component (a decision tree, logistic/linear regression, or a comparably simple model).
- Generate a synthetic workload or trace that includes a deliberate shift partway through (the access pattern, request pattern, or load pattern changes).
- Measure and report how both the classical algorithm and the learned component respond to that shift.

4. Choose One Track

Track 1 - Learned Page Replacement (Memory Management)

Background

Page replacement policies such as FIFO and LRU assume a fairly stable access pattern. Real workloads rarely stay stable for long.

What to implement

- Implement FIFO, LRU, and Optimal (Belady's) page replacement.

Learned / adaptive component

Using simple features such as recency and access frequency, train a lightweight classifier (e.g., a decision tree or logistic regression) that predicts a good eviction candidate, and compare it against the classical policies.

Shift / stress test

Generate a synthetic page-access trace where the first half follows a locality-heavy / sequential pattern and the second half switches to a random or bursty pattern.

What to measure and report

- Hit ratio and page-fault count for each policy, before and after the shift.
- A short discussion of which policy degrades the most when the pattern changes, and why.

Track 2 - Disk Scheduling: Learned Scheduler Selector

Background

No single disk-scheduling algorithm (FCFS, SCAN, C-SCAN, SSTF) is best for every workload; the right choice depends on the request pattern.

What to implement

- Implement FCFS, SCAN, C-SCAN, and SSTF.

Learned / adaptive component

Build a small classifier that looks at a window of incoming requests (e.g., variance of track positions, burstiness) and predicts which scheduler will perform best, along with a confidence score for that prediction.

Shift / stress test

Test across synthetic workload types (sequential, random, bursty) and a timeline where the workload type changes partway through.

What to measure and report

- Total seek time achieved by each classical scheduler versus the one the classifier selected.
- Whether the classifier's confidence score is actually higher on the cases it gets right — i.e., is it well calibrated?

Track 3 - Deadlock: Early Risk Classifier

Background

The Banker's Algorithm and resource-allocation-graph cycle detection both confirm deadlock only after the full request sequence is known.

What to implement

- Implement the Banker's Algorithm safety check and resource-allocation-graph cycle detection.

Learned / adaptive component

Randomly generate resource-request sequences and label each as safe or unsafe using the full algorithm. Train a simple classifier on partial (in-progress) sequences to flag “risky” trajectories before the full check would normally complete.

Shift / stress test

Vary the number of processes and resource types, and include sequences that only become unsafe late in the trace.

What to measure and report

- How many steps earlier, on average, the classifier flags risk compared to the full algorithm.
- False-positive and false-negative rates for the early-warning classifier.

Track 4 - Virtualization / Cloud: Predictive VM Placement

Background

Static bin-packing placement (best-fit, first-fit) allocates VMs to hosts based on the load at allocation time, and does not anticipate future demand.

What to implement

- Implement static best-fit and first-fit VM-to-host placement.

Learned / adaptive component

Build a simple regression model that estimates each VM’s near-future load from its recent history, and use that prediction to place VMs proactively rather than reactively.

Shift / stress test

Use a synthetic multi-VM load trace that includes a load spike or demand shift partway through.

What to measure and report

- Host utilization balance, number of overload / SLA-violation events, and migration count, for static versus predictive placement.

Optional / Bonus Track - Explanation Confidence (up to +10 points)

For any of the four tracks above, use a free-tier LLM API (or a simple rule-based template generator if no API is available) to produce a short natural-language explanation of the algorithm’s decision at each step, along with a self-rated confidence score. Compare that confidence score against whether the decision was actually correct. This track is optional and does not replace the base implementation — it is an extension for groups who want to go further.

5. Deliverables

- GitHub repository containing: source code, a README.md (setup instructions, how to run, and a short summary of results), and a results/ folder with at least one chart or table.
- A short written report, 2–3 pages (PDF), containing: problem framing, implementation summary, experimental setup, results, and a short analysis of what changed under the shift.
- Printed copy of the report, submitted in class.

- A 3–5 minute in-class walkthrough / demo of the code.

6. Grading Rubric

Component	Points
Correct implementation of the classical algorithm(s)	30
Design and justification of the learned / adaptive component	25
Experimental design (workload generation, shift scenario)	20
Results and analysis what changed under the shift, and why	15
Code quality, documentation, and GitHub repository	10
Total	100

7. Submission Instructions

- Share your GitHub repository link through the course submission channel by the deadline above.
- Submit the soft copy (pdf) of your report to class within the submission date.

8. Academic Integrity

Code, results, and analysis must be your own work. You may use standard libraries, course materials, and AI coding assistants for implementation help, but you must understand and be able to explain every part of your submission. Copying another group's code, workload generator, or report is an academic integrity violation.